

Fast and Accurate Domain Adaptation for Irregular Tensor Decomposition

Junghun Kim
Seoul National University
Seoul, South Korea
bandalg97@snu.ac.kr

Ka Hyun Park
Seoul National University
Seoul, South Korea
kahyun.park@snu.ac.kr

Jun-Gi Jang
University of Illinois
Urbana-Champaign
Urbana, Illinois, USA
jungi@illinois.edu

U Kang
Seoul National University
Seoul, South Korea
ukang@snu.ac.kr

Abstract

Given an irregular tensor from a newly emerging domain, how can we quickly and accurately capture its patterns utilizing existing irregular tensors in multiple domains? The problem is of great importance for various tasks such as finding patterns of a new disease using pre-existing diseases data. This is challenging as new target tensors have limited information due to their recent emergence. Thus, carefully utilizing the existing source tensors for analyzing the target tensor is helpful. PARAFAC2 decomposition is a strong tool for finding the patterns of irregular tensors, and the patterns are used in many applications such as missing value prediction and anomaly detection. However, previous PARAFAC2-based works cannot adaptably handle newly emerging target tensors utilizing the source tensors.

In this work, we propose META-P2, a fast and accurate domain adaptation method for irregular tensor decomposition. META-P2 generates a meta factor matrix from the multiple source domains, by domain adaptation and meta-update steps. META-P2 quickly and accurately finds the patterns of the new irregular tensor utilizing the meta factor matrix. Extensive experiments on real-world datasets show that META-P2 achieves the best performance in various downstream tasks including missing value prediction and anomaly detection tasks.

CCS Concepts

• **Computing methodologies** → **Factorization methods**; **Transfer learning**.

Keywords

multi-domain, irregular tensor, tensor decomposition

ACM Reference Format:

Junghun Kim, Ka Hyun Park, Jun-Gi Jang, and U Kang. 2024. Fast and Accurate Domain Adaptation for Irregular Tensor Decomposition. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)*, August 25–29, 2024, Barcelona, Spain. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3637528.3671670>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
KDD '24, August 25–29, 2024, Barcelona, Spain.

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0490-1/24/08
<https://doi.org/10.1145/3637528.3671670>

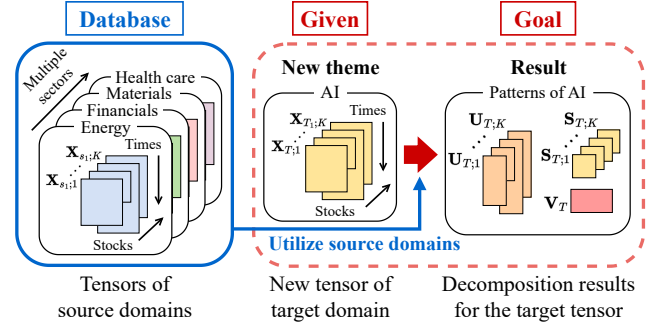


Figure 1: An example of domain adaptation for irregular tensors. We are given a stock database, which is represented as irregular tensors of multiple domains. Each domain represents a specific sector, and each slice $X_{d;k}$ denotes time-varying stock information (such as opening and closing prices) of k -th stock in sector d . Then, we encounter a new unseen themed-stocks (target tensor). The goal is to find patterns within this new tensor through tensor decomposition exploiting data from multiple source domains.

1 Introduction

How can we quickly and accurately capture the patterns of a new irregular tensor from a target domain utilizing irregular tensors from various source domains? Many real-world data often take the form of irregular tensors across various domains. For example, stocks from diverse industries are represented as irregular tensors in multiple domains. Despite their distinct listing periods, these tensors share a common set of features, such as the opening or closing price. In this scenario, the objective is to efficiently and accurately capture patterns within the stocks of a recently emerged industry by leveraging pre-existing stock data (see Figure 1). The challenge involves addressing how to quickly identify the patterns of a new irregular tensor within the target domain, utilizing insights gained from irregular tensors sourced from various domains. The problem includes many other scenarios such as finding new disease patterns using pre-existing diseases data as well as finding action patterns of a new person using the records of multiple people.

To find patterns of high-dimensional data, decomposition-based methods have been actively studied in data mining area [24, 36?]. Among the various methods, PARAFAC2 decomposition methods are widely used for finding the patterns of irregular tensors by representing them with low-rank latent factor matrices. The

low-rank latent factor matrices are used for many applications including missing value prediction [3], anomaly detection [5], failure prediction [43], and image recommendation [30].

However, existing PARAFAC2-based decomposition methods are not easy to be directly used in our problem. There are two challenges for them: 1) the limited information of target tensor due to their recent emergence, and 2) disparities between the domains (e.g., stock movements differ across various industries). A naive approach would be to conduct PARAFAC2 decomposition only on the new target tensor. This neglects valuable information from the source domains, and leads to an inaccurate decomposition of the target tensor. Another approach would perform PARAFAC2 decomposition on source and target tensors simultaneously by combining all tensors. However, this approach fails to find accurate latent factor matrices of the target tensor if the source and target domains have different properties. Furthermore, the approach should decompose the tensors from entire domains whenever a new target tensor is given, thus is computationally inefficient. Therefore, it is important to carefully extract useful information from source domains that help analyze the target tensor in advance.

In this work, we propose META-P2, a fast and accurate domain adaptation method for irregular tensor decomposition. META-P2 carefully learns a general latent factor matrix from multiple source domains based on the meta-learning approach (see Section 2.3 for details). Then the latent factor matrix is used for analyzing a new target tensor. This process enables us to effectively transfer the general knowledge from source domains to the target domain, leading to accurate decomposition of the target tensor.

META-P2 successfully tackles the challenges emerged in previous approaches. First, META-P2 efficiently supplements the limited information of the target tensor by proactively extracting useful information from the source tensors. Unlike existing methods, META-P2 does not require tensors from entire domains to be decomposed together when a new target tensor emerges. Second, META-P2 accurately decomposes the target tensor in scenarios where the source and target domains exhibit distinct characteristics. This is achieved by extracting *general* and *easily adaptable* information from the source domains, instead of domain-specific one.

Our contributions are summarized as follows:

- **Method.** We propose META-P2, a fast and accurate domain adaption method for decomposing irregular tensors. META-P2 effectively finds the patterns of a new irregular tensor via meta-learning.
- **Theory.** We give theoretical analysis on the properties of META-P2, including the time complexities, and the relation between the meta-update step of META-P2 and the update step of the previous meta-learning approach.
- **Experiments.** Extensive experiments on real-world datasets show that META-P2 achieves the best performance in missing value prediction and anomaly detection problems.

The rest of this paper is organized as follows. Section 2 gives preliminaries and a formal definition of our problem. Section 3 proposes META-P2 and analyzes its time complexity. Section 4 shows experimental results, and Section 5 introduces related works. We conclude the paper in Section 6. The code and datasets are available at <https://github.com/snudatalab/Meta-P2>.

Table 1: Symbols.

Symbol	Description
$\{\mathbf{X}_k\}_{k=1}^K$	irregular tensor ($\in \mathbb{R}^{I_k \times J \times K}$ for $k = 1, \dots, K$)
$\mathbf{X}_k$	k -th slice of $\{\mathbf{X}_k\}_{k=1}^K$ ($\in \mathbb{R}^{I_k \times J}$)
$\mathbf{U}_k, \mathbf{S}_k, \mathbf{Q}_k$	factor matrices of the k -th slice $\mathbf{X}_k$
$\mathbf{H}, \mathbf{V}$	factor matrices of $\{\mathbf{X}_k\}_{k=1}^K$
$\{\mathbf{X}_{d;k}\}_{k=1}^K$	irregular tensor of domain d
$\mathbf{X}_{d;k}$	k -th slice of $\{\mathbf{X}_{d;k}\}_{k=1}^K$ ($\in \mathbb{R}^{I_k \times J}$)
$\mathbf{U}_{d;k}, \mathbf{S}_{d;k}, \mathbf{Q}_{d;k}$	factor matrices of the k -th slice $\mathbf{X}_{d;k}$
$\mathbf{H}_d, \mathbf{V}_d$	factor matrices of $\{\mathbf{X}_{d;k}\}_{k=1}^K$
$\mathbf{X}_{(n)}$	mode- n matricization of $\mathbf{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N}$
$\odot$	Khatri-Rao product
$*$	Hadamard product
R	target rank of decomposition

2 Preliminaries and Problem

We describe tensor notations and operations, PARAFAC2, and meta-learning. We then formally define the problem.

2.1 Tensor Notations and Operations

We provide the symbols used in this paper in Table 1.

Irregular Tensor. We denote an irregular tensor as a set $\{\mathbf{X}_k\}_{k=1}^K$ where $\mathbf{X}_k \in \mathbb{R}^{I_k \times J}$ is the k -th slice of the tensor, and K is the number of slices. Note that the row sizes I_k of the slices are different for all k . An irregular tensor in domain d is denoted as $\{\mathbf{X}_{d;k}\}_{k=1}^K$. We denote $\{\mathbf{X}_{d;k}\}_{k=1}^K \in \mathbb{R}^{I_k \times J \times K}$ as $\{\mathbf{X}_d\}$ in this paper for brevity.

Mode- n Matricization. Given a tensor $\mathbf{X} \in \mathbb{R}^{I_1 \times \dots \times I_N}$, we denote mode- n matricization of $\mathbf{X}$ as $\mathbf{X}_{(n)}$. The resulting unfolded matrix is of size $(I_n, \prod_{m=1, m \neq n}^N I_m)$. The operation arranges the mode- n fibers to be the columns of the resulting matrix.

Khatri-Rao Product. Khatri-Rao product of two matrices $\mathbf{A} \in \mathbb{R}^{p \times r}$ and $\mathbf{B} \in \mathbb{R}^{q \times r}$, which is denoted by $\mathbf{A} \odot \mathbf{B} \in \mathbb{R}^{pq \times r}$, is computed as $\mathbf{A} \odot \mathbf{B} = [\mathbf{A}(:, 1) \otimes \mathbf{B}(:, 1)] \cdots [\mathbf{A}(:, r) \otimes \mathbf{B}(:, r)]$ where $\otimes$ is Kronecker product and $|$ is the horizontal concatenation. The Kronecker product $\mathbf{A}(:, k) \otimes \mathbf{B}(:, k)$ of k -th column is computed as $[\mathbf{A}(1, k)\mathbf{B}(:, k)^T \mid \cdots \mid \mathbf{A}(p, k)\mathbf{B}(:, k)^T]^T$.

Hadamard Product. Given two matrices $\mathbf{A}$ and $\mathbf{B}$ of the same size, the Hadamard product $\mathbf{A} * \mathbf{B}$ is a matrix of the same size as the operands, with elements given by $(\mathbf{A} * \mathbf{B})(i, j) = \mathbf{A}(i, j)\mathbf{B}(i, j)$.

2.2 PARAFAC2 Decomposition

PARAFAC2 decomposition analyzes an irregular tensor by approximating it with a multiplication of latent factor matrices. The objective of PARAFAC2 is to minimize the following objective function:

$$\min_{\{\mathbf{U}_k\}, \{\mathbf{S}_k\}, \mathbf{V}} \sum_{k=1}^K \|\mathbf{X}_k - \mathbf{U}_k \mathbf{S}_k \mathbf{V}^T\|_F^2 \quad (1)$$

where $\mathbf{X}_k \in \mathbb{R}^{I_k \times J}$ is a k -th slice of the original irregular tensor $\{\mathbf{X}_k\}_{k=1}^K \in \mathbb{R}^{I_k \times J \times K}$, and $\mathbf{U}_k \in \mathbb{R}^{I_k \times R}$, $\mathbf{S}_k \in \mathbb{R}^{R \times R}$, and $\mathbf{V} \in \mathbb{R}^{J \times R}$ are the decomposed factor matrices of $\mathbf{X}_k$ with rank R . Note that $\mathbf{V}$ is shared along the slices.

To ensure the uniqueness of the results, Harshman [14] replaced $\mathbf{U}_k$ with $\mathbf{Q}_k \mathbf{H}$ where $\mathbf{Q}_k \in \mathbb{R}^{I_k \times R}$ is a column orthogonal matrix,

and $\mathbf{H} \in \mathbb{R}^{R \times R}$ is a common matrix for all the slices. Then the objective function of PARAFAC2 is reformulated as

$$\min_{\{\mathbf{Q}_k\}, \{\mathbf{S}_k\}, \mathbf{H}, \mathbf{V}} \sum_{k=1}^K \|\mathbf{X}_k - \mathbf{Q}_k \mathbf{H} \mathbf{S}_k \mathbf{V}^\top\|_F^2. \quad (2)$$

ALS (Alternating Least Square) is commonly used for optimizing Equation (2) due to its fast speed and accuracy [46]. CP-ALS, which uses ALS for PARAFAC2 decomposition, iteratively updates each factor matrix while fixing the other ones. A single iteration of CP-ALS updates factor matrices by running Equations (3)-(7) once. CP-ALS first finds $\mathbf{Q}_k$ while fixing $\mathbf{H}$, $\mathbf{S}_k$, and $\mathbf{V}$ for all k :

$$\mathbf{A}_k \Sigma \mathbf{B}_k^\top \leftarrow \mathbf{X}_k \mathbf{V} \mathbf{S}_k \mathbf{H}^\top \quad (3)$$

$$\mathbf{Q}_k \leftarrow \mathbf{A}_k \mathbf{B}_k^\top \quad (4)$$

where Equation (3) is the truncated SVD with rank R . Then, CP-ALS updates $\mathbf{H}$, $\{\mathbf{S}_k\}$, and $\mathbf{V}$ as follows:

$$\mathbf{W} \leftarrow \mathbf{Y}_{(3)} (\mathbf{V} \odot \mathbf{H}) (\mathbf{V}^\top \mathbf{V} * \mathbf{H}^\top \mathbf{H})^\dagger \quad (5)$$

$$\mathbf{H} \leftarrow \mathbf{Y}_{(1)} (\mathbf{W} \odot \mathbf{V}) (\mathbf{W}^\top \mathbf{W} * \mathbf{V}^\top \mathbf{V})^\dagger \quad (6)$$

$$\mathbf{V} \leftarrow \mathbf{Y}_{(2)} (\mathbf{W} \odot \mathbf{H}) (\mathbf{W}^\top \mathbf{W} * \mathbf{H}^\top \mathbf{H})^\dagger \quad (7)$$

where the k -th row of $\mathbf{W}_s \in \mathbb{R}^{K \times R}$ is the diagonal entries of $\mathbf{S}_k$, $\mathbf{Y} \in \mathbb{R}^{R \times J \times K}$ is a constructed tensor from slices $\mathbf{Y}_k$ for all k , and $\mathbf{Y}_k \in \mathbb{R}^{R \times J}$ is $\mathbf{Q}_k^\top \mathbf{X}_k$. For each k , $\mathbf{S}_k$ and $\mathbf{U}_k$ are updated as $\text{diag}(\mathbf{W}(k, :))$ and $\mathbf{Q}_k \mathbf{H}$, respectively. CP-ALS repeats Equations (3)-(7) until the results converge.

2.3 Model-Agnostic Meta-Learning (MAML)

Meta-learning is a powerful framework to address the problem of limited data in many machine learning areas. Meta-learning trains a general model with multiple given tasks so that it adapts to a new problem quickly, instead of training task-specific models. Previous meta-learning methods train the general model using gradient information from the given tasks [16, 39]. Among them, MAML [10] has been widely used due to its robustness and accuracy.

MAML trains a model on multiple given tasks $\mathcal{T}_d$ for $d = 1, \dots, D$, with the goal of finding a set of initial parameters θ_{meta} that can be fine-tuned to perform well on a new target task quickly and accurately. Note that the objective of θ_{meta} is not to perform well on the given tasks, but to quickly adapt to a new unseen target task.

For every given task $\mathcal{T}_d$, MAML adapts the meta model parameters θ_{meta} to each $\mathcal{T}_d$ by performing a single gradient descent update with the objective function $\mathcal{L}_{\mathcal{T}_d}$:

$$\theta'_d \leftarrow \theta_{\text{meta}} - \alpha \nabla_{\theta_{\text{meta}}} \mathcal{L}_{\mathcal{T}_d}(f_{\theta_{\text{meta}}}) \quad (8)$$

where θ'_d denotes the adapted θ_{meta} to each $\mathcal{T}_d$. This simulates the adaptation process of θ_{meta} to a new target task with a few gradient-descent updates.

MAML then updates θ_{meta} following the mean of gradients $\nabla_{\theta} \mathcal{L}_{\mathcal{T}_d}(f_{\theta'_d})$ for $d = 1, \dots, D$ as follows:

$$\theta_{\text{meta}} \leftarrow \theta_{\text{meta}} - \beta \nabla_{\theta_{\text{meta}}} \sum_{d=1}^D \mathcal{L}_{\mathcal{T}_d}(f_{\theta'_d}). \quad (9)$$

Note that the meta-gradient is a gradient in terms of θ_{meta} whereas the loss function is parameterized by θ'_d . This enables θ_{meta} to contain easily transplantable information from all tasks $\mathcal{T}_d$. We denote $\nabla_{\theta_{\text{meta}}} \sum_{d=1}^D \mathcal{L}_{\mathcal{T}_d}(f_{\theta'_d})$, which is a specially designed gradient for

meta-learning, as *meta-gradient* in the rest of this paper. We repeat the process until the maximum iteration is reached.

2.4 Problem Definition

We formally define the problem in Problem 1.

PROBLEM 1 (DOMAIN ADAPTATION FOR IRREGULAR TENSOR DECOMPOSITION). *We are given a set $\mathcal{S}$ of source domains. For each domain $s \in \mathcal{S}$, there is an irregular tensor $\{\mathbf{X}_s\}$. Then, we have a new irregular tensor $\{\mathbf{X}_T\}$ from an unseen target domain T . The problem aims to quickly find the latent factor matrices of $\{\mathbf{X}_T\}$ that reconstruct $\{\mathbf{X}_T\}$ accurately exploiting the tensors of source domains.*

The objective function $\mathcal{L}$ of the problem is as follows:

$$\min_{\theta_T} \mathcal{L}(\{\mathbf{X}_T\}; \theta_T) = \sum_{k=1}^K \|\mathbf{X}_{T;k} - \mathbf{U}_{T;k} \mathbf{S}_{T;k} \mathbf{V}_T^\top\|_F^2 \quad (10)$$

where $\mathbf{X}_{T;k} \in \mathbb{R}^{I_k \times J}$ is a k -th slice of $\{\mathbf{X}_T\} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_K \times J}$, and $\theta_T = \{\{\mathbf{U}_T\}, \{\mathbf{S}_T\}, \mathbf{V}_T\}$. Note that the knowledge of source domains is utilized for optimizing the objective function in Equation (10).

3 Proposed Method

We propose META-P2, our fast and accurate domain adaptation method for irregular tensors. The main challenges and our approaches are as follows:

- (1) **How can we utilize given source tensors for decomposing a new tensor in a target domain quickly and accurately?** We learn a general *meta factor matrix* from given source domains, which is easily transferred to the new target domain (Section 3.1).
- (2) **How can we find the meta factor matrix from source domains which is easily transferred to the target domain?** We perform *meta-learning-based PARAFAC2 decomposition* on source domains. This is achievable by carefully modeling the gradients of reconstruction error in terms of the meta factor matrix (Section 3.2).
- (3) **How can we utilize the trained meta factor when decomposing a new tensor of a target domain?** We adapt the meta factor to the target domain while preventing the decomposed results of the target domain from completely forgetting the information of source domains (Section 3.3).

3.1 Meta Factor Matrix

The objective of META-P2 is to quickly find accurate latent factor matrices of a new irregular tensor $\{\mathbf{X}_T\}$ of the target domain T , using the given irregular tensors $\{\mathbf{X}_s\}$ in multiple source domains $s \in \mathcal{S}$ (see Section 2.4 for a formal problem definition and its objective function). Specifically, we aim to accurately decompose the target tensor $\{\mathbf{X}_T\}$ with limited information in a few ALS iteration updates. To do this, it is important to define the information from source domains that helps decompose the target tensor. If the information is slice-specific or domain-specific, it cannot directly be used for analyzing the new unseen target tensor.

For each tensor $\{\mathbf{X}_s\}$ in a source domain s , the decomposed results are represented by the multiplication of three components: $\{\mathbf{U}_{s;k}\}$, $\{\mathbf{S}_{s;k}\}$, and $\mathbf{V}_s$. Each k -th slice $\mathbf{X}_{s;k}$ of $\{\mathbf{X}_s\}$ is decomposed into $\mathbf{U}_{s;k} \mathbf{S}_{s;k} \mathbf{V}_s$. Note that $\mathbf{V}_s$ contains representative information

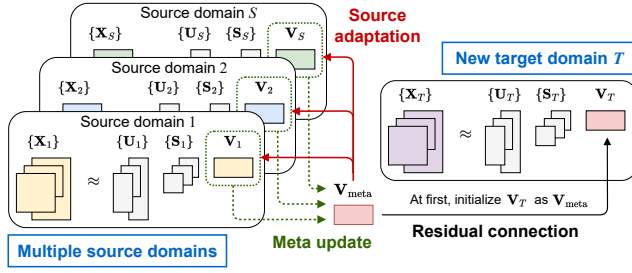


Figure 2: Overall process of META-P2. We train the easily adaptable meta factor $\mathbf{V}_{\text{meta}}$ from multiple source tensors $\{\mathbf{X}_s\}$ for $s \in \{1, \dots, S\}$. This is done by iteratively performing the source adaptation and meta-update steps. We utilize the trained $\mathbf{V}_{\text{meta}}$ for decomposing a new target tensor $\{\mathbf{X}_T\}$.

of domain s since $\mathbf{V}_s$ is used for constructing every slice of $\{\mathbf{X}_s\}$, while $\{\mathbf{U}_{s;k}\}$ and $\{\mathbf{S}_{s;k}\}$ are specific to each slice.

We propose to share $\mathbf{V}_s$ along the source domains, which we denote as $\mathbf{V}_{\text{meta}}$, and utilize it for analyzing the target domain. The intuition is to learn *general* information from the source domains by forcing $\mathbf{V}_{\text{meta}}$ to reconstruct every source tensor well. A challenge is that naively sharing $\mathbf{V}_{\text{meta}}$ only aims to reconstruct the source tensors, and fails to achieve accurate results on the target domain when the target has unseen properties in source domains.

To address this, META-P2 enforces $\mathbf{V}_{\text{meta}}$ to possess an additional property: *easily adaptable*. We learn $\mathbf{V}_{\text{meta}}$ to quickly and accurately adapt to any new domain, not only to the given source domains. This is done by training $\mathbf{V}_{\text{meta}}$ based on the meta-learning scheme. As a result, META-P2 learns *general* and *easily adaptable* meta factor matrix $\mathbf{V}_{\text{meta}}$ from the source tensors.

The overall process of META-P2 is illustrated in Figure 2. We first compute the meta factor matrix $\mathbf{V}_{\text{meta}}$ from multiple source domains. Then we leverage the trained $\mathbf{V}_{\text{meta}}$ for decomposing a new target tensor $\{\mathbf{X}_T\}$. This enables us to effectively utilize useful information of source domains for analyzing the target tensor without decomposing all the domains together.

3.2 Decomposition on Source Domains

META-P2 learns the *general* and *easily adaptable* meta factor $\mathbf{V}_{\text{meta}}$ from the multiple source tensors based on the meta-learning scheme. The meta-learning aims to train a general model that adapts to a new task quickly and accurately utilizing multiple source tasks. The meta-learning based decomposition of META-P2 in the source domains consists of two steps: a) source adaptation and b) meta-update steps. The source adaptation step adapts $\mathbf{V}_{\text{meta}}$ to every source domain. Using the adapted results, the meta-update step finds $\mathbf{V}_{\text{meta}}$ that is prepared for adaptation to any new domain, rather than accurate reconstruction of the source tensors. We repeat those two steps until the maximum iteration is reached. Algorithm 1 shows the overall process of decomposition in source domains.

Using the objective function $\mathcal{L}$ of META-P2 in Equation (10), the source adaptation of $\mathbf{V}_{\text{meta}}$ for each source domain s formulated as:

$$\mathbf{V}_s = \mathbf{V}_{\text{meta}} - \eta \nabla_{\mathbf{V}_{\text{meta}}} \sum_{k=1}^K \|\mathbf{X}_{s;k} - \mathbf{U}_{s;k} \mathbf{S}_{s;k} \mathbf{V}_{\text{meta}}^\top\|_F^2 \quad (11)$$

Algorithm 1: Computing the meta factor matrix from multiple source domains.

input : Irregular tensors $\{\mathbf{X}_{s;k}\}_{k=1}^K \in \mathbb{R}^{I_k \times J \times K}$ from all source domains $s = 1, \dots, S$
output : Meta factor matrix $\mathbf{V}_{\text{meta}} \in \mathbb{R}^{J \times R}$
parameters : Target rank R

- 1 Randomly initialize $\mathbf{W}_s \in \mathbb{R}^{R \times R}$, $\mathbf{H}_s \in \mathbb{R}^{R \times R}$ for all $s = 1, \dots, S$, and $\mathbf{V}_{\text{meta}}$;
- 2 **repeat**
 - /* Source adaptation step */
 - 3 **for** $s=1, \dots, S$ **do**
 - 4 Adapt $\mathbf{V}_{\text{meta}}$ to domain s by a single ALS iteration, and save the result as $\mathbf{V}_s$; // Equations (15), (16)
 - 5 **end**
 - /* Meta-update step */
 - 6 Update $\mathbf{V}_{\text{meta}}$ along the *meta-gradient*, which is computed by aggregating all $\mathbf{V}'_s$; // Equation (21)
- 7 **until** the maximum iteration is reached;

where η is a learning rate. To remove the constant term $\mathbf{U}_{s;k} \mathbf{S}_{s;k}$ in front of $\mathbf{V}_{\text{meta}}$, we rewrite $\mathcal{L}(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}})$ as follows:

$$\mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}}) = \sum_{k=1}^K \|\mathbf{E}_{s;k} \mathbf{X}_{s;k} - \mathbf{V}_{\text{meta}}^\top\|_F^2. \quad (12)$$

where $\mathbf{E}_{s;k} = K(\sum_{j=1}^K \mathbf{S}_{s;j}^\top \mathbf{U}_{s;j}^\top \mathbf{U}_{s;j} \mathbf{S}_{s;j})^{-1} \mathbf{S}_{s;k}^\top \mathbf{U}_{s;k}^\top$. Note that minimizing $\mathcal{L}_V$ in Equation (12) is equivalent to minimizing $\mathcal{L}$ in Equation (10) for training $\mathbf{V}_{\text{meta}}$ (see Appendix A.1 for proof). Then the source adaptation step of META-P2 is expressed as follows:

$$\mathbf{V}_s = \mathbf{V}_{\text{meta}} - \eta \nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}}). \quad (13)$$

The meta-update step of $\mathbf{V}_{\text{meta}}$ is as follows:

$$\mathbf{V}_{\text{meta}} \leftarrow \mathbf{V}_{\text{meta}} - \eta \nabla_{\mathbf{V}_{\text{meta}}} \sum_{s \in S} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s). \quad (14)$$

However, CP-ALS algorithm is widely used for learning the latent factors of irregular tensors [1, 19, 31] compared to the gradient-descent one. This is mainly because the ALS-based decomposition gives accurate factors in a few iterations and guarantees the uniqueness in the decomposed results [14]. In the following Sections 3.2.1 and 3.2.2, we describe how we design the gradient-based meta-learning by the ALS-based meta-learning for META-P2.

3.2.1 Source adaptation step. In the source adaptation step, we adapt $\mathbf{V}_{\text{meta}}$ to each source domain by performing a single ALS iteration instead of the single gradient-based update in Equation (13). The idea stems from the observation that both gradient-based and ALS-based training finds the optimal parameter by gradually minimizing the objective function. This idea also aligns with the typical meta-learning scenario which constrains the numbers of gradient-descent updates to adapt the meta model parameters to each given task. To perform the ALS-based adaptation of $\mathbf{V}_{\text{meta}}$, we first update $\mathbf{W}_s$ and $\mathbf{H}_s$ using Equations (5) and (6), respectively:

$$\begin{aligned} \mathbf{W}_s &= (\mathbf{Y}_s)_{(3)} (\mathbf{V}_{\text{meta}} \odot \mathbf{H}_s) (\mathbf{V}_{\text{meta}}^\top \mathbf{V}_{\text{meta}} * \mathbf{H}_s^\top \mathbf{H}_s)^\dagger \\ \mathbf{H}_s &= (\mathbf{Y}_s)_{(1)} (\mathbf{W}_s \odot \mathbf{V}_{\text{meta}}) (\mathbf{W}_s^\top \mathbf{W}_s * \mathbf{V}_{\text{meta}}^\top \mathbf{V}_{\text{meta}})^\dagger \end{aligned} \quad (15)$$

where $\mathbf{Y}_s \in \mathbb{R}^{R \times J \times K}$ is constructed from slices $\mathbf{Y}_{s;k} = \mathbf{Q}_{s;k}^\top \mathbf{X}_{s;k}$ for all $k = 1, \dots, K$, and $\mathbf{Q}_{s;k} \in \mathbb{R}^{I_k \times R}$ is computed using Equation (4). The k -th row of $\mathbf{W}_s \in \mathbb{R}^{K \times R}$ corresponds to the diagonal entries of $\mathbf{S}_k$. Then we adapt $\mathbf{V}_{\text{meta}}$ to each domain s using Equation (7):

$$\mathbf{V}_s = (\mathbf{Y}_s)_{(2)} (\mathbf{W}_s \odot \mathbf{H}_s) (\mathbf{W}_s^\top \mathbf{W}_s * \mathbf{H}_s^\top \mathbf{H}_s)^\dagger. \quad (16)$$

Note that we express the gradient-based source adaptation step in Equation (13) by an ALS-based adaptation in Equation (16).

3.2.2 Meta-update step. In the meta-update step, we update the meta factor matrix $\mathbf{V}_{\text{meta}}$ to perform well after adapting to any new target domain, rather than generally performing well across all source domains. Our approach for designing the gradient-based meta-update in Equation (14) by an ALS-based meta-update for META-P2 is to express the *meta-gradient* $\nabla_{\mathbf{V}_{\text{meta}}} \sum_{s \in \mathcal{S}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$ using the factors computed by ALS iterations. However, directly differentiating $\mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$ in terms of $\mathbf{V}_{\text{meta}}$ is challenging since $\mathbf{V}_s$ is not parameterized by $\mathbf{V}_{\text{meta}}$.

To address this, we break down each $\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$ of the meta-gradient into two gradients $\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}})$ and $\nabla_{\mathbf{V}_s} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$. In the following, we first describe how we express the two gradients using only the factors computed by ALS iterations, and then we show the relationship between the meta-gradient and the two gradients.

Two Gradients. Let $\mathbf{V}'_s$ denote $\mathbf{V}_s$ adapted to domain s again:

$$\mathbf{V}'_s = \mathbf{V}_s - \eta \nabla_{\mathbf{V}_s} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s) \quad (17)$$

$$= (\mathbf{Y}_s)_{(2)} (\mathbf{W}'_s \odot \mathbf{H}'_s) (\mathbf{W}'_s{}^\top \mathbf{W}'_s * \mathbf{H}'_s{}^\top \mathbf{H}'_s)^\dagger \quad (18)$$

where $\mathbf{W}'_s$ and $\mathbf{H}'_s$ denote $\mathbf{W}_s$ and $\mathbf{H}_s$ after performing an additional ALS updates in Equation (15) while replacing $\mathbf{V}_{\text{meta}}$ with $\mathbf{V}_s$. The second equality uses our approximation in the source adaptation step that a single gradient-descent update is equal to a single ALS iteration update (see Equations (13) and (16)). Then the two gradients $\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}})$ and $\nabla_{\mathbf{V}_s} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$ are simply expressed by $(\mathbf{V}_{\text{meta}} - \mathbf{V}_s)/\eta$ and $(\mathbf{V}_s - \mathbf{V}'_s)/\eta$ using Equations (13) and (17), respectively. Note that $\mathbf{V}_s$ and $\mathbf{V}'_s$ are computed by the ALS iterations in Equations (16) and (18).

Relationship between Meta-gradient and the Two Gradients. Lemma 3.1 shows the relationship between each term $\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$ of the meta-gradient and the two gradients $\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}})$ and $\nabla_{\mathbf{V}_s} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$.

LEMMA 3.1. *Each term $\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$ of the meta-gradient is proportional to the sum of the two gradients $\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}})$ and $\nabla_{\mathbf{V}_s} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$:*

$$\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s) = C (\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}}) + \nabla_{\mathbf{V}_s} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)) \quad (19)$$

where $C = (1 - 2\eta K)^2 / (2(1 - \eta K))$ is a constant. η is the learning rate and K is the number of slices in each source domain (see Appendix A.2 for proof).

Using Lemma 3.1, we derive Theorem 3.2 which expresses the meta-update step of META-P2 in terms of $\mathbf{V}'_s$ that is computed by the ALS iteration in Equation (18).

Algorithm 2: Tensor decomposition of META-P2 on a new target tensor exploiting the meta factor matrix.

input : Irregular tensor $\mathbf{X}_T \in \mathbb{R}^{I_k \times J \times K}$, and a trained meta factor $\mathbf{V}_{\text{meta}} \in \mathbb{R}^{J \times R}$

output : Decomposed matrices $\mathbf{U}_{T;k} \in \mathbb{R}^{I_k \times R}$ and $\mathbf{S}_{T;k} \in \mathbb{R}^{R \times R}$ for all k , and $\mathbf{V}_T$

parameters: Target rank R

- 1 Randomly initialize $\mathbf{H}_T \in \mathbb{R}^{R \times R}$, and $\mathbf{S}_{T;k}$ for all k ;
- 2 Initialize $\mathbf{V}_T$ with $\mathbf{V}_{\text{meta}}$;
- 3 **repeat**
- 4 Update $\{\mathbf{Q}_T\}, \mathbf{H}_T, \mathbf{W}_T, \mathbf{V}_T$ using Equations (3)-(7);
- 5 $\mathbf{V}_T \leftarrow (\mathbf{V}_T + \mathbf{V}_{\text{meta}})/2$; // residual connection
- 6 **until** the maximum iteration is reached;
- 7 **for** $k=1, \dots, K$ **do**
- 8 $\mathbf{S}_{T;k} \leftarrow \text{diag}(\mathbf{W}_T(k, :));$
- 9 $\mathbf{U}_{T;k} \leftarrow \mathbf{Q}_{T;k} \mathbf{H}_T$;
- 10 **end**

THEOREM 3.2. *The meta-update step of META-P2 in Equation (14) is expressed in terms of $\mathbf{V}'_s$ that is computed using the ALS iterations:*

$$\mathbf{V}_{\text{meta}} \leftarrow \mathbf{V}_{\text{meta}} + C \sum_{s \in \mathcal{S}} (\mathbf{V}'_s - \mathbf{V}_{\text{meta}}). \quad (20)$$

$C = (1 - 2\eta K)^2 / (2(1 - \eta K))$ is a constant with learning rate η and the number K of slices in each domain (see Appendix A.3 for proof).

We choose η that satisfies $C = 1/|\mathcal{S}|$. As a result, the meta-update step of $\mathbf{V}_{\text{meta}}$ is simply expressed by the mean of $\mathbf{V}'_s$ for all source domains $s \in \mathcal{S}$ as follows:

$$\mathbf{V}_{\text{meta}} \leftarrow \mathbf{V}_{\text{meta}} + \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} (\mathbf{V}'_s - \mathbf{V}_{\text{meta}}) = \frac{1}{|\mathcal{S}|} \sum_{s \in \mathcal{S}} \mathbf{V}'_s. \quad (21)$$

Note that we express the meta-update step of META-P2 in Equation (14) using only $\mathbf{V}'_s$, which is computed by performing ALS iterations. We learn $\mathbf{V}_{\text{meta}}$ from the source tensors by repeatedly performing the source adaptation step in Equation (16) and the meta-update step in Equation (21).

3.3 Inference on Target Domain

META-P2 analyzes a new tensor $\mathbf{X}_T$ of the target domain T using the patterns of source domains. The patterns are represented as a meta factor $\mathbf{V}_{\text{meta}}$. The main idea is to initialize the latent factor matrix $\mathbf{V}_T$ of target domain T with $\mathbf{V}_{\text{meta}}$, and find the adapted factor matrices to the target domain by running a few ALS iterations. The decomposed results are utilized for performing many downstream tasks on the target domain including missing value prediction, anomaly detection, feature extraction, etc.

For performing a downstream task, overfitting the latent factors to the target domain degrades the performance when there are only a few entries in the target tensor. To prevent forgetting the trained patterns $\mathbf{V}_{\text{meta}}$ of source domains while decomposing the target tensor, we exploit a residual connection between $\mathbf{V}_{\text{meta}}$ and $\mathbf{V}_T$.

When decomposing a new target tensor, abnormal values such as anomalies and missing entries deteriorate the quality of the decomposed results. Since a single ALS step of META-P2 reconstructs

the target tensor accurately using the general information $\mathbf{V}_{\text{meta}}$ of source domains, META-P2 imputes the abnormal values with the corresponding reconstructed values after a single ALS step. For example, when there are anomalies in the target tensor, META-P2 replaces the detected anomalous entries with the reconstructed values after a single ALS step on the target tensor.

3.4 Theoretical Analysis

We theoretically analyze the time complexity of META-P2. Specifically, we show that the meta-learning-based decomposition of META-P2 has the same complexity to that of a naive PARAFAC2.

LEMMA 3.3. *The decomposition of META-P2 on source domains (Algorithm 1) has the same time complexity as that of a naive PARAFAC2 decomposition on the source domains (see Appendix A.4 for proof).*

LEMMA 3.4. *Given the meta factor matrix $\mathbf{V}_{\text{meta}}$, the time complexity of META-P2 for finding latent factors of a target tensor (Algorithm 2) is the same as that of performing PARAFAC2 decomposition on the target tensor (see Appendix A.5 for proof).*

4 Experiments

We perform experiments to answer the following questions.

- Q1. Missing Value Prediction.** How accurately does META-P2 predict missing values of a new target tensor using existing tensors in multiple source domains (Section 4.2)?
- Q2. Anomaly Detection.** Does META-P2 show superior performance than the baselines for detecting anomalies of a new target tensor exploiting the source tensors (Section 4.3)?
- Q3. Domain Adaptation Speed.** How fast does META-P2 adapt to the new target domain (Section 4.4)?
- Q4. Ablation Study.** Does each step of META-P2 contribute to the performance of the downstream task (Section 4.5)?
- Q5. Case Study.** What are the discoveries of analyzing real world data using META-P2 (Section 4.6)?

4.1 Experimental Settings

We present our experimental setup including datasets and baselines. All experiments are done with a single GTX 3080.

Datasets. We use five real-world datasets from two distinct fields: stock and human action. We summarize the statistics of datasets in Table 2.

Nasdaq, S&P500, and Korea stock are stock datasets which contain selected features of daily stock prices in Nasdaq, S&P 500, and Korea stock market, respectively. We use 1) the opening, closing, highest, and lowest prices, and 2) trading volume of the day as features. Stocks are grouped into sectors, and we treat their sectors as domains. This is because the statistics of stocks, such as price or trading volume, are similar within each sector, but differ among sectors. In each sector, the data are in the form of (*day, feature, stock*); a stock corresponds to a slice matrix of size (*day, feature*).

NATOPS and Cricket are human activity datasets measured with various sensors. NATOPS is a collection of 20 people repeatedly performing 24 aircraft handling actions. We treat the people as domains since each person has different physical conditions and unique habits. For each person, the actions are in the form of (*timeframe, sensor, action*); an action corresponds to a slice matrix whose

Table 2: Summary of datasets.

Name	# Domain	Max I_k	(J, K)	# Non-zero	Type
Nasdaq ¹	11	12,709	(6, 11)	2,742K	stock
S&P500 ¹	11	13,321	(6, 13)	7,318K	stock
Korea stock ²	11	3,089	(6, 10)	2,038K	stock
NATOPS ³	20	2,009	(77, 24)	42,720K	HAR
Cricket ⁴	12	1,197	(6, 10)	1,292K	HAR

¹ <https://kaggle.com/datasets/paultimothymooney/stock-market-data>

² <https://github.com/jungijang/KoreaStockData>

³ <https://github.com/yalesong/natops>

⁴ <https://timeseriesclassification.com>

form is (*timeframe, sensor*). Cricket is a collection of four umpires performing twelve signals in the cricket game, each with ten repetitions. We consider each signal type, such as No Ball or Dead Ball, as a domain for the Cricket dataset. This is because the sensor values are different depending on the type of signal due to the differences in the type of movements involved. For each type of signal, the signals are in the form of (*timeframe, sensor, person*); a person corresponds to a slice matrix, and its form is (*timeframe, sensor*).

If the number of slices in a specific domain is extremely large, the decomposed results that do not consider the diversity of domains are biased towards the domain. This is unfair to the competitors including PARAFAC2 and PARAFAC2-I. Thus, we randomly sample K slices for each domain where K is the number of slices of the domain with the least slices. In stock datasets, scales vary largely across the columns; e.g., scales of the price and trading volume are different. Thus, we perform L2 normalization for each j -th column $\mathbf{X}(:, j)$ of a slice matrix $\mathbf{X}$: $\mathbf{X}(i, j) = \mathbf{X}(i, j) / \|\mathbf{X}(:, j)\|_2$.

Baselines. META-P2 aims to decompose the target tensor quickly and accurately utilizing the information of given source tensors. Thus, we compare META-P2 with previous approaches for irregular tensor decomposition. We do not compare META-P2 with COPA [1] and SPARTan [31] that aim to improve the scalability of the original PARAFAC2 since our objective is to find accurate latent factors of the target tensor with limited number of slices.

PARAFAC2 [14] performs PARAFAC2 decomposition on the whole domains including source and target. The method optimizes the latent factor matrices based on ALS approach. PARAFAC2-T performs PARAFAC2 decomposition on the target domain only. PARAFAC2-I initializes the factor matrix $\mathbf{V}$ of the target domain with the result of a naive PARAFAC2 decomposition on source domains. AO-ADMM [33] is a recent state-of-the-art decomposition method for irregular tensors. AO-ADMM employs an alternating optimization (AO) approach in conjunction with the alternating direction method of multipliers (ADMM) for fitting PARAFAC2.

Evaluation and Settings. To evaluate the quality of decomposed results of META-P2, we perform two downstream tasks: missing value prediction and anomaly detection. For each dataset, we set the last domain (in alphabetical order) as the unseen target domain, and the rest as source domains. For example, in stock datasets, we align sectors alphabetically and use the last sector, *Utilities* in our case, as the target domain. We randomly split the data of the target domain into training and test missing (anomalous) entries with the missing (anomaly) ratio. In the missing value prediction task, we

Table 3: The missing value prediction error of META-P2 and baselines. The lowest errors are in bold. META-P2 shows the best prediction performance in all cases.

Models	Missing Value Prediction									
	Missing ratio $r_m = 0.1$					Missing ratio $r_m = 0.3$				
	Nasdaq	S&P500	Korea stock	NATOPS	Cricket	Nasdaq	S&P500	Korea stock	NATOPS	Cricket
PAFARAC2	0.3149 ± 0.0412	0.3551 ± 0.0360	0.4085 ± 0.0095	0.3881 ± 0.0157	1.3469 ± 0.1304	0.4998 ± 0.0593	0.4948 ± 0.0608	0.4894 ± 0.0207	0.6172 ± 0.0081	1.1392 ± 0.0602
PARAFAC2-I	0.9905 ± 0.0033	1.0075 ± 0.0035	0.9933 ± 0.0024	0.8119 ± 0.0041	1.0203 ± 0.0056	1.0054 ± 0.0014	1.0035 ± 0.0006	1.0087 ± 0.0015	0.8498 ± 0.0019	1.0212 ± 0.0050
PARAFAC2-T	0.9902 ± 0.0025	1.0094 ± 0.0012	0.9974 ± 0.0015	0.8243 ± 0.0026	1.0235 ± 0.0041	1.0097 ± 0.0020	1.0041 ± 0.0004	1.0086 ± 0.0022	0.8558 ± 0.0013	1.0218 ± 0.0032
AO-ADMM	0.5263 ± 0.0038	0.3761 ± 0.0019	0.4645 ± 0.0134	0.9081 ± 0.0268	0.7630 ± 0.0649	0.5944 ± 0.0033	0.4246 ± 0.0052	0.6129 ± 0.0097	1.3270 ± 0.0235	1.0767 ± 0.0071
META-P2 (proposed)	0.2884 ± 0.0287	0.3237 ± 0.0278	0.3849 ± 0.0260	0.2907 ± 0.0042	0.9024 ± 0.0627	0.3896 ± 0.0279	0.3852 ± 0.0295	0.4666 ± 0.0228	0.3890 ± 0.0044	0.9078 ± 0.0335

Table 4: Anomaly detection performance of META-P2 and baselines in terms of F1 score. Bold numbers denote the best performance. META-P2 shows the best detection performance among the methods in most of the cases.

Models	Anomaly Detection									
	Anomaly ratio $r_a = 0.05$					Anomaly ratio $r_a = 0.1$				
	Nasdaq	S&P500	Korea stock	NATOPS	Cricket	Nasdaq	S&P500	Korea stock	NATOPS	Cricket
PAFARAC2	0.0069 ± 0.0002	0.0041 ± 0.0002	0.0012 ± 0.0005	0.0297 ± 0.0001	0.4094 ± 0.0203	0.0145 ± 0.0004	0.0066 ± 0.0004	0.0013 ± 0.0006	0.0648 ± 0.0006	0.4742 ± 0.0137
PARAFAC2-I	0.2400 ± 0.0199	0.2482 ± 0.0719	0.4594 ± 0.0530	0.2365 ± 0.0036	0.0867 ± 0.0150	0.2167 ± 0.0222	0.1985 ± 0.0439	0.3830 ± 0.0532	0.2510 ± 0.0019	0.1299 ± 0.0112
PARAFAC2-T	0.2301 ± 0.0397	0.1920 ± 0.0348	0.4017 ± 0.0613	0.2256 ± 0.0033	0.0758 ± 0.0096	0.2096 ± 0.0233	0.1691 ± 0.0177	0.3255 ± 0.0462	0.2354 ± 0.0031	0.1280 ± 0.0104
AO-ADMM	0.1293 ± 0.0061	0.3591 ± 0.0162	0.5100 ± 0.0118	0.1089 ± 0.0052	0.0123 ± 0.0021	0.1972 ± 0.0066	0.4049 ± 0.0060	0.5400 ± 0.0122	0.1161 ± 0.0032	0.0324 ± 0.0024
META-P2 (proposed)	0.3508 ± 0.0097	0.4656 ± 0.0121	0.5959 ± 0.0165	0.2960 ± 0.0017	0.0752 ± 0.0094	0.3912 ± 0.0092	0.4993 ± 0.0107	0.6282 ± 0.0133	0.3298 ± 0.0009	0.1292 ± 0.0085

use the normalized error as the evaluation metric; given an original tensor $\mathbf{X}$ and the reconstructed one $\hat{\mathbf{X}}$, the normalized error r_{err} is:

$$r_{\text{err}} = \frac{\sum_{k=1}^K \sum_{(i,j) \in \Omega_k} \|\mathbf{X}_k(i,j) - \hat{\mathbf{X}}_k(i,j)\|_F^2}{\sum_{k=1}^K \sum_{(i,j) \in \Omega_k} \|\mathbf{X}_k(i,j)\|_F^2} \quad (22)$$

where $\mathbf{X}_k$ is the k -th frontal slice, $\hat{\mathbf{X}}_k$ is the reconstructed one of $\mathbf{X}_k$ with target rank R , and Ω_k is the set of test entries of the k -th slice matrix. In the anomaly detection task, we use the F1 score as the evaluation metric. For each dataset, we set the target rank R as the minimum mode length of the target tensor, and the maximum number of iterations as 10. We run all experiments ten times with random seeds, and report the average and standard deviation removing the highest and the lowest results.

4.2 Missing Value Prediction (Q1)

We present the missing value prediction errors of META-P2 and baselines in Table 3. META-P2 consistently outperforms competitors in various scenarios, achieving the lowest reconstruction error across different missing ratios $r_m \in \{0.1, 0.3\}$. This indicates that it is important to carefully use the information of source domains when decomposing the target domain. Furthermore, PARAFAC2-T which focuses solely on the new target tensor presents lower accuracy than other methods including PARAFAC2 and PARAFAC2-I. This is because solely reconstructing the target tensor with limited information incurs overfitting problem, showing inaccurate prediction for the missing entries.

4.3 Anomaly Detection (Q2)

To evaluate the anomaly detection performance of META-P2, we randomly select anomalous entries in the target tensor $\mathbf{X}_T$ with the anomaly ratio $r_a \in \{0.05, 0.1\}$. Then we change the values of the entries by dividing them with a constant $\psi = 2$, to make them anomalies. We detect K anomalies by finding top- K entries with the highest error between the anomaly-injected tensor and our reconstructed one.

Table 4 presents that META-P2 gives the highest F1 score with large margin in most of the cases. This indicates that META-P2 successfully discovers the factor matrices that contribute to accurately detecting anomalies in a new target tensor.

In Cricket, PARAFAC2 shows better performance than META-P2. The main reason is that Cricket has extremely small number of entries in the target domain, with excessively small number of anomalies. Thus, PARAFAC2 easily neglects the anomalous entries while reconstructing the tensor of entire domains, showing higher reconstruction performance on those anomaly values. However, PARAFAC2 completely fails to detect anomalies in other datasets, and does not perform well for missing value prediction task with markedly slower speed (see Figure 4).

4.4 Domain Adaptation Speed (Q3)

META-P2 initializes the latent factor matrix $\mathbf{V}_T$ of the target domain T with the meta factor matrix $\mathbf{V}_{\text{meta}}$. To show that $\mathbf{V}_{\text{meta}}$ adapts to the target domain quickly and effectively, we compare the missing value prediction performance of META-P2 with differently initialized $\mathbf{V}_T$. PARAFAC2-initialized method initializes $\mathbf{V}_T$ with the decomposed result of naive PARAFAC2 decomposition on the source domains. Random-initialized method initializes $\mathbf{V}_T$ randomly. We adopt the imputation process of missing entries and the residual connection of META-P2 to those baselines. We report the performance of the methods over the ALS iterations in Figure 3. We also show the runtime of them per iteration in Figure 4.

Figure 3 shows that META-P2 effectively reduces the prediction error with only a few ALS iterations where the runtime per ALS iteration is the same for all initializing methods (see Figure 4). The prediction errors of other methods are generally monotonous and poor compared to META-P2. This shows that initializing $\mathbf{V}_T$ with $\mathbf{V}_{\text{meta}}$ assists in discovering accurate latent factor matrices of the target tensor quickly. Moreover, PARAFAC2-initialized consistently shows poor performance. This implies that simply tuning the decomposed results to a new target domain cannot consider the difference between source and target domains.

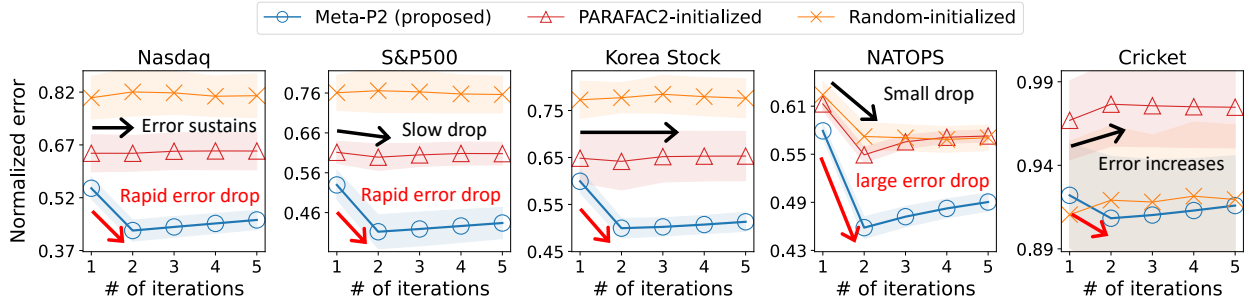


Figure 3: The missing value prediction performance over the ALS iterations. We compare META-P2 with two other initialization approaches of V_T for a target domain T . Note that the error of META-P2 drops rapidly, achieving the best prediction performance among the initialization methods. The errors of other baselines drop slowly, or even increase in Cricket.

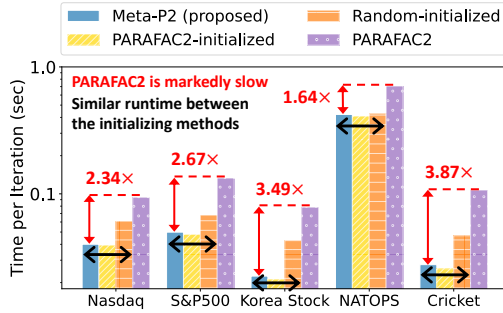


Figure 4: Running time for each ALS iteration on a target domain. Note that META-P2 achieves up to 3.87 $\times$ faster speed than PARAFAC2 which decomposes the entire tensors in multiple domains whenever a new target tensor emerges. Meta-P2 shows similar runtime to other initializing methods.

4.5 Ablation Study (Q4)

We provide an ablation study on the missing value prediction with respect to the effectiveness of each module in META-P2. META-P2-meta is META-P2 without the meta-learning-based decomposition on the source domains; it performs a naive PARAFAC2 decomposition instead of meta-learning-based decomposition on the source domains, and use the naively decomposed result when adapting to a new target domain. META-P2-imp and META-P2-res are META-P2 without considering the missing values by imputation, and META-P2 without the residual connection of the meta factor, respectively.

Table 5 shows that each module of META-P2 effectively reduces the prediction error. Specifically, META-P2-meta shows poor performance compared to the proposed META-P2 in every case. This indicates that the meta-learning-based decomposition on source domains effectively enhances the quality of decomposition result of a new target domain.

4.6 Case Study (Q5)

We showed that META-P2 successfully detects or predicts the abnormal values through Sections 4.2 and 4.3. Our further question is, *how is an anomalous point related to a real-world event?* To answer this, we present discoveries of analyzing historical prices of S&P500. We

Table 5: Ablation study on missing value prediction problem. The lowest errors are in bold. Note that each module of META-P2 improves the performance of missing value prediction.

Missing Value Prediction (missing ratio $r_m = 0.5$)				
Datasets	MTD-imp	MTD-res	MTD-meta	META-P2 (proposed)
Nasdaq	0.9240 $\pm$ 0.0093	0.7079 $\pm$ 0.0433	0.6526 $\pm$ 0.0396	0.5102 $\pm$ 0.0261
SP500	0.8721 $\pm$ 0.0152	0.6671 $\pm$ 0.0302	0.6053 $\pm$ 0.0209	0.4712 $\pm$ 0.0251
Korea stock	0.9100 $\pm$ 0.0274	0.7242 $\pm$ 0.0401	0.6530 $\pm$ 0.0417	0.5595 $\pm$ 0.0172
NATOPS	0.7887 $\pm$ 0.0012	0.6105 $\pm$ 0.0024	0.5728 $\pm$ 0.0055	0.5097 $\pm$ 0.0042
Cricket	1.0119 $\pm$ 0.0045	0.9702 $\pm$ 0.0262	0.9763 $\pm$ 0.0161	0.9297 $\pm$ 0.0205

compose a new target domain by selecting ten largest AI-themed stocks within S&P500 dataset (see Figure 1), while keeping other settings consistent with those described in Section 4.1. We detect five abnormal days by finding the days with the top-5 largest errors between the true stock features and the reconstructed ones.

Figure 5 compares the abnormal days detected by META-P2 and PARAFAC2-T in IBM and Netflix. We observe that the detected days of META-P2 are closely related to real-world events, showing its efficacy in interpreting anomalies. We also note that the detected points of PARAFAC2-T are densely distributed compared to META-P2, failing to detect several real-world events that META-P2 has successfully identified.

5 Related Works

We review previous PARAFAC2 decomposition approaches for irregular tensors and meta learning methods.

Tensor decomposition. Tensor decomposition methods are applied to many real-world tasks including model compression [32, 38, 41, 45], anomaly detection [9, 27, 37, 47], forecasting [8, 26, 35], answering time range queries [2, 17, 18], and missing value prediction [25]. However, those decomposition methods for regular tensors assume that every dimension has the same length across the slices, which is not true for irregular tensors. Recently, neural tensor factorization methods have been actively studied in recommender systems [4, 42]. The main drawback of the neural tensor models compared to ALS-based tensor decomposition methods is its computational complexity. Neural tensor models involve training a neural network, which can be computationally intensive, especially for large datasets and complex architectures.

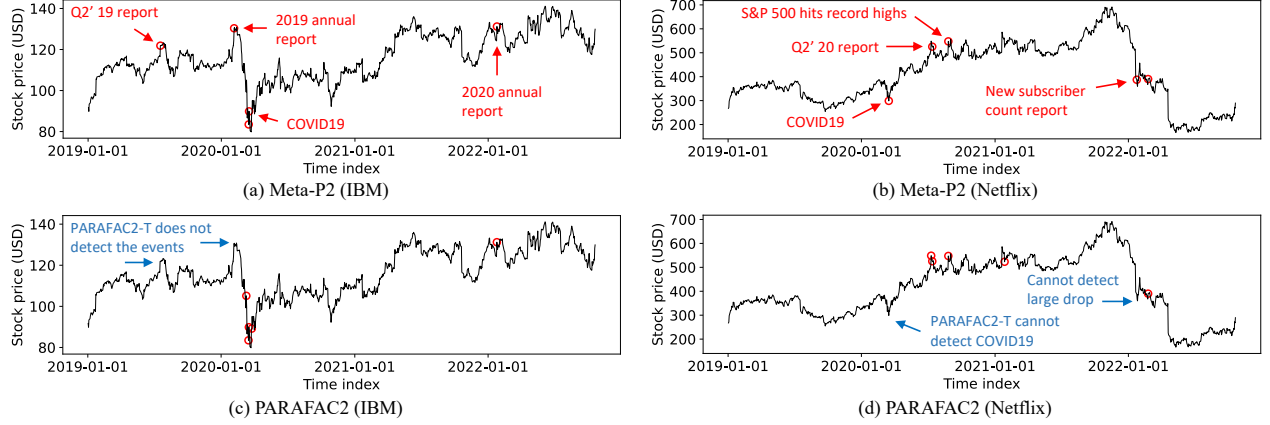


Figure 5: Top-5 anomaly detection results of META-P2 on (a) IBM and (b) Netflix, and PARAFAC2-T on (c) IBM and (d) Netflix. Note that the detected anomalies (red circles) of META-P2 are closely related to real-world events during the corresponding periods. PARAFAC2-T detect densely distributed abnormal points, leading to the oversight of various real-world events.

PARAFAC2 decomposition. Many works have been studied to find patterns of irregular tensors based on PARAFAC2 [6, 29, 30, 40, 43]. DPar2 [19, 20] is a preprocessing algorithm for decomposing irregular dense tensors. The method effectively compresses each slice matrix of a given irregular tensor by careful reordering of computations. SPARTan [31] is a scalable PARAFAC2 decomposition algorithm for irregular sparse tensors. COPA [1] is an extension of SPARTan, which improves the performance of it with additional constraints. To find the latent factors in large streaming tensors, SPADE [12] defines a non-temporal factor and processes the newly incoming part using the accelerated MTTKRP [31]. Dash [21] efficiently finds the latent factors by avoiding naive computations involved with old data when new data are given. However, those methods have limitations in decomposing a new target tensor using given tensors in multiple domains since they assume all tensors are in the same domain.

Various task-specific irregular tensor decomposition methods have been studied to improve the performance of their target tasks. To predict missing entries in irregular tensors, Atom [22] reformulates the objective function to completely exclude missing values, ensuring more accurate predictions. LogPar [44] addresses the missing values by modeling the irregular tensor with a Bernoulli distribution, parameterized by an underlying real-valued tensor, and approximates this tensor using a positive-unlabeled learning loss function. However, those task-specific methods (e.g., missing value prediction) are applicable only to specific tensors (e.g., tensors with missing values). Meta-P2 is a task-agnostic decomposition method (general framework) that can be applied to many downstream tasks.

Meta-learning. Many meta-learning methods have been proposed to deal with the problem of limited data in many machine learning areas [11, 15, 34]. FOMAML (first-order MAML) [28] is a variation of MAML that uses first-order gradient information to update the model’s parameters. However, the previous works on meta-learning are not directly applicable for PARAFAC2 since they update model parameters based on gradient descent update whereas the decomposition methods update them through CP-ALS.

Applications of meta-learning. Previous works have used the meta-learning algorithm to solve few show learning problems. Chien et al. [7] employed meta-learning algorithm for hyperparameter optimization in dialogue system. Gupta et al. [13] used unsupervised meta-learning algorithm for reinforcement learning. Lian et al. [23] applied meta-learning algorithm to neural architecture search to learn a meta-architecture that is able to adapt to a new task quickly through a few gradient steps. To the best of our knowledge, META-P2 is the first domain adaptation method for decomposing irregular tensors based on meta-learning approach.

6 Conclusion

We propose META-P2, a fast and accurate domain adaptation method for decomposing irregular tensors. META-P2 obtains a meta factor matrix from existing tensors in multiple source domains, and the meta factor helps find accurate patterns of a new target tensor quickly. This is achieved by training the meta factor to possess two key properties: 1) *generality* by sharing the meta factor along the source domains, and 2) *easy adaptability* by ALS-based meta-learning. Extensive experiments on real-world datasets show that META-P2 achieves the best performance in missing value prediction and anomaly detection problems. Future works include extending META-P2 for other decomposition methods beyond PARAFAC2.

Acknowledgments

This work was supported by the National Research Foundation of Korea (NRF) funded by MSIT (2022R1A2C3007921). This work was also supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) [No.2022-0-00641, XVoice: Multi-Modal Voice Meta Learning], [No.RS-2021-II211343, Artificial Intelligence Graduate School Program (Seoul National University)], and [No.RS-2021-II212068, Artificial Intelligence Innovation Hub (Artificial Intelligence Institute, Seoul National University)]. U Kang is the corresponding author.

References

- [1] Ardavan Afshar, Ioakeim Perros, Evangelos E. Papalexakis, Elizabeth Searles, Joyce C. Ho, and Jimeng Sun. 2018. COPA: Constrained PARAFAC2 for Sparse & Large Datasets. In *CIKM*.
- [2] Dawon Ahn, Jun-Gi Jang, and U Kang. 2022. Time-aware tensor decomposition for sparse tensors. *Mach. Learn.* (2022).
- [3] Dawon Ahn, Seyun Kim, and U Kang. 2021. Accurate Online Tensor Factorization for Temporal Tensor Streams with Missing Values. In *CIKM*.
- [4] Huiyuan Chen and Jing Li. 2020. Neural tensor model for learning multi-aspect factors in recommender systems. In *IJCAI*.
- [5] Peter A. Chew, Brett W. Bader, Stephen Helmreich, Ahmed Abdelali, and Stephen J. Verzi. 2011. An information-theoretic, vector-space-model approach to cross-language information retrieval. *Nat. Lang. Eng.* 17, 1 (2011), 37–70.
- [6] Peter A. Chew, Brett W. Bader, Tamara G. Kolda, and Ahmed Abdelali. 2007. Cross-language information retrieval using PARAFAC2. In *SIGKDD*. ACM.
- [7] Jen-Tzung Chien and Wei Xiang Liew. 2019. Meta Learning for Hyperparameter Optimization in Dialogue System. In *Interspeech*.
- [8] Beyza Ermis, Evrim Acar, and Ali Taylan Cemgil. 2015. Link prediction in heterogeneous data via generalized coupled tensor factorization. *Data Min. Knowl. Discov.* (2015).
- [9] Hadi Fanaee-T and João Gama. 2016. Tensor-based anomaly detection: An interdisciplinary survey. *Knowl. Based Syst.* (2016).
- [10] Chelsea Finn, Pieter Abbeel, and Sergey Levine. 2017. Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks. In *ICML*.
- [11] Chelsea Finn, Aravind Rajeswaran, Sham Kakade, and Sergey Levine. 2019. Online meta-learning. In *ICML*.
- [12] Ekta Gujral, Georgios Theodorou, and Evangelos E. Papalexakis. 2020. SPADE: Streaming PARAFAC2 DEcomposition for Large Datasets. In *SDM*.
- [13] Abhishek Gupta, Benjamin Eysenbach, Chelsea Finn, and Sergey Levine. 2018. Unsupervised Meta-Learning for Reinforcement Learning. *CoRR* (2018).
- [14] R. A. Harshman. 1972. PARAFAC2: Mathematical and Technical Notes. In *UCLA Working Papers in Phonetics*.
- [15] Timothy M. Hospedales, Antreas Antoniou, Paul Micaelli, and Amos J. Storkey. 2022. Meta-Learning in Neural Networks: A Survey. *IEEE Trans. Pattern Anal. Mach. Intell.* (2022).
- [16] Muhammad Abdullah Jamal and Guo-Jun Qi. 2019. Task Agnostic Meta-Learning for Few-Shot Learning. In *CVPR*.
- [17] Jun-Gi Jang, Dongjin Choi, Jinhong Jung, and U Kang. 2018. Zoom-SVD: Fast and Memory Efficient Method for Extracting Key Patterns in an Arbitrary Time Range. In *CIKM*.
- [18] Jun-Gi Jang and U Kang. 2021. Fast and Memory-Efficient Tucker Decomposition for Answering Diverse Time Range Queries. In *KDD*.
- [19] Jun-Gi Jang and U Kang. 2022. DPar2: Fast and Scalable PARAFAC2 Decomposition for Irregular Dense Tensors. In *ICDE*.
- [20] Jun-Gi Jang and U Kang. 2023. Static and Streaming Tucker Decomposition for Dense Tensors. *ACM Trans. Knowl. Discov. Data* 17, 5 (2023), 66:1–66:34.
- [21] Jun-Gi Jang, Jeongyoung Lee, Yong-chan Park, and U Kang. 2023. Fast and Accurate Dual-Way Streaming PARAFAC2 for Irregular Tensors - Algorithm and Application. In *KDD*.
- [22] Jun-Gi Jang, Jeongyoung Lee, Jiwon Park, and U Kang. 2022. Accurate PARAFAC2 decomposition for temporal irregular tensors with missing values. In *Big Data*.
- [23] Dongze Lian, Yin Zheng, Yintao Xu, Yanxiang Lu, Leyu Lin, Peilin Zhao, Junzhou Huang, and Shenghua Gao. 2020. Towards Fast Adaptation of Neural Architectures with Meta Learning. In *ICLR*.
- [24] Yu-Ru Lin, Jimeng Sun, Paul C. Castro, Ravi B. Konuru, Hari Sundaram, and Aisling Kelliher. 2009. MetaFac: community discovery via relational hypergraph factorization. In *SIGKDD*.
- [25] Hanpeng Liu, Yaguang Li, Michael Tsang, and Yan Liu. 2019. Costco: A neural tensor completion model for sparse tensors. In *SIGKDD*.
- [26] Xiaoyang Ma, Lan Zhang, Lan Xu, Zhicheng Liu, Ge Chen, Zhili Xiao, Yang Wang, and Zhengtao Wu. 2019. Large-scale User Visits Understanding and Forecasting with Deep Spatial-Temporal Tensor Factorization Framework. In *SIGKDD*.
- [27] Maxwell McNeil, Carolina Mattsson, Frank W Takes, and Petko Bogdanov. 2023. CADENCE: Community-Aware Detection of Dynamic Network States. In *SDM*.
- [28] Alex Nichol, Joshua Achiam, and John Schulman. 2018. On First-Order Meta-Learning Algorithms. *CoRR* (2018).
- [29] Yannis Panagakis and Constantine Kotropoulos. 2011. Automatic music tagging via PARAFAC2. In *ICASSP*. IEEE, 481–484.
- [30] Evangelia Pantraki and Constantine Kotropoulos. 2015. Automatic image tagging and recommendation via PARAFAC2. In *MLSP*.
- [31] Ioakeim Perros, Evangelos E. Papalexakis, Fei Wang, Richard W. Vuduc, Elizabeth Searles, Michael Thompson, and Jimeng Sun. 2017. SPARTan: Scalable PARAFAC2 for Large & Sparse Data. In *SIGKDD*.
- [32] Anh Huy Phan, Konstantin Sobolev, Konstantin Sozkyin, Dmitry Ermilov, Julia Gusak, Petr Tichavský, Valeriy Glukhov, Ivan V. Oseledets, and Andrzej Cichocki. 2020. Stable Low-Rank Tensor Decomposition for Compression of Convolutional Neural Network. In *ECCV*.
- [33] Marie Roald, Carla Schenker, Vince D Calhoun, Tulay Adali, Rasmus Bro, Jeremy E Cohen, and Evrim Acar. 2022. An AO-ADMM approach to constraining PARAFAC2 on all modes. *SIAM Journal on Mathematics of Data Science* (2022).
- [34] Nicolas Schweighofer and Kenji Doya. 2003. Meta-learning in Reinforcement Learning. *Neural Networks* (2003).
- [35] Alessandro Spelta. 2017. Financial market predictability with tensor decomposition and links forecast. *Appl. Netw. Sci.* 2 (2017), 7.
- [36] Stephan Spiegel, Jan Hendrik Clausen, Sahin Albayrak, and Jérôme Kunegis. 2011. Link Prediction on Evolving Data Using Tensor Factorization. In *PAKDD*.
- [37] Leo Tisljaric, Sofia Fernandes, Tonci Caric, and João Gama. 2020. Spatiotemporal Traffic Anomaly Detection on Urban Road Network Using Tensor Decomposition Method. In *Discovery Science - 23rd International Conference*.
- [38] Andros Tjandra, Sakriani Sakti, and Satoshi Nakamura. 2018. Tensor Decomposition for Compressing Recurrent Neural Network. In *IJCNN*.
- [39] Ricardo Vilalta and Youssef Drissi. 2002. A Perspective View and Survey of Meta-Learning. *Artif. Intell. Rev.* (2002).
- [40] Barry M Wise, Neal B Gallagher, and Elaine B Martin. 2001. Application of PARAFAC2 to fault detection and diagnosis in semiconductor etch. *Journal of Chemometrics: A Journal of the Chemometrics Society* (2001).
- [41] Bijiao Wu, Dingheng Wang, Guangshe Zhao, Lei Deng, and Guoqi Li. 2020. Hybrid tensor decomposition in neural network compression. *Neural Networks* 132 (2020), 309–320.
- [42] Xian Wu, Baoxu Shi, Yuxiao Dong, Chao Huang, and Nitesh V Chawla. 2019. Neural tensor factorization for temporal interaction learning. In *WSDM*.
- [43] Kejing Yin, Ardavan Afshar, Joyce C. Ho, William K. Cheung, Chao Zhang, and Jimeng Sun. 2020. LogPar: Logistic PARAFAC2 Factorization for Temporal Binary Data with Missing Values. In *SIGKDD*.
- [44] Kejing Yin, Ardavan Afshar, Joyce C Ho, William K Cheung, Chao Zhang, and Jimeng Sun. 2020. LogPar: Logistic PARAFAC2 factorization for temporal binary data with missing values. In *SIGKDD*.
- [45] Miao Yin, Siyu Liao, Xiao-Yang Liu, Xiaodong Wang, and Bo Yuan. 2020. Compressing Recurrent Neural Networks Using Hierarchical Tucker Tensor Decomposition. *CoRR abs/2005.04366* (2020).
- [46] Dave Zachariah, Martin Sundin, Magnus Jansson, and Saikat Chatterjee. 2012. Alternating Least-Squares for Low-Rank Matrix Reconstruction. *IEEE Signal Process. Lett.* 19, 4 (2012), 231–234.
- [47] Xing Zhang, Gongjian Wen, and Wei Dai. 2016. A Tensor Decomposition-Based Anomaly Detection Algorithm for Hyperspectral Image. *IEEE Trans. Geosci. Remote. Sens.* (2016).

A Appendix

A.1 Relation of Equation (12) and Equation (10)

LEMMA A.1. Suppose we are given a loss function $\mathcal{L}(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}}) = \sum_{k=1}^K \|\mathbf{X}_{s;k} - \mathbf{U}_{s;k} \mathbf{S}_{s;k} \mathbf{V}_{\text{meta}}^\top\|_F^2$. Then finding optimal $\mathbf{V}_{\text{meta}}$ by minimizing $\mathcal{L}(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}})$ is equivalent to finding it by minimizing the following loss function:

$$\mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}}) = \sum_{k=1}^K \|\mathbf{E}_{s;k} \mathbf{X}_{s;k} - \mathbf{V}_{\text{meta}}^\top\|_F^2. \quad (23)$$

where $\mathbf{E}_{s;k} = \mathbf{K}(\sum_{j=1}^K \mathbf{S}_{s;j}^\top \mathbf{U}_{s;j}^\top \mathbf{U}_{s;j} \mathbf{S}_{s;j})^{-1} \mathbf{S}_{s;k}^\top \mathbf{U}_{s;k}^\top$.

PROOF. To find optimal $\mathbf{V}_{\text{meta}}$ that minimizes a given loss, we differentiate the loss in terms of $\mathbf{V}_{\text{meta}}$ and find $\mathbf{V}_{\text{meta}}$ where the derivative equals zero.

We first find $\mathbf{V}_{\text{meta}}$ that minimizes the original loss $\mathcal{L}(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}})$:

$$\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}}) \quad (24)$$

$$= -2 \sum_{k=1}^K (\mathbf{X}_{s;k}^\top - \mathbf{V}_{\text{meta}} \mathbf{S}_{s;k}^\top \mathbf{U}_{s;k}^\top) \mathbf{U}_{s;k} \mathbf{S}_{s;k} = 0 \quad (25)$$

$$\Leftrightarrow \mathbf{V}_{\text{meta}} \sum_{k=1}^K \mathbf{S}_{s;k}^\top \mathbf{U}_{s;k}^\top \mathbf{U}_{s;k} \mathbf{S}_{s;k} = \sum_{k=1}^K \mathbf{X}_{s;k}^\top \mathbf{U}_{s;k} \mathbf{S}_{s;k} \quad (26)$$

$$\Leftrightarrow \mathbf{V}_{\text{meta}} = \sum_{k=1}^K \mathbf{X}_{s;k}^\top \mathbf{U}_{s;k} \mathbf{S}_{s;k} (\sum_{j=1}^K \mathbf{S}_{s;j}^\top \mathbf{U}_{s;j}^\top \mathbf{U}_{s;j} \mathbf{S}_{s;j})^{-1} \quad (27)$$

We then discover $\mathbf{V}_{\text{meta}}$ that minimizes the reformulated loss $\mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}})$:

$$\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}}) \quad (28)$$

$$= -2 \sum_{k=1}^K (\mathbf{X}_{s;k}^\top \mathbf{E}_{s;k}^\top - \mathbf{V}_{\text{meta}}) = 0 \quad (29)$$

$$\Leftrightarrow \mathbf{V}_{\text{meta}} = \frac{1}{K} \sum_{k=1}^K \mathbf{X}_{s;k}^\top \mathbf{E}_{s;k}^\top. \quad (30)$$

Since $\mathbf{E}_{s;k} = K(\sum_{j=1}^K \mathbf{S}_{s;j}^\top \mathbf{U}_{s;j}^\top \mathbf{U}_{s;j} \mathbf{S}_{s;j})^{-1} \mathbf{S}_{s;k}^\top \mathbf{U}_{s;k}^\top$, Equation (30) is equal to Equation (27) which ends the proof. $\square$

A.2 Proof of Lemma 3.1

PROOF. We denote $\mathbf{E}_{s;k} \mathbf{X}_{s;k}$ in Equation (12) as $\mathbf{Z}_{s;k}$ for simplicity. Then the loss function $\mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}})$ is written as:

$$\mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}}) = \sum_{k=1}^K \|\mathbf{Z}_{s;k} - \mathbf{V}_{\text{meta}}^\top\|_F^2. \quad (31)$$

The gradient $\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}})$ is computed as:

$$\begin{aligned} \nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}}) &= \nabla_{\mathbf{V}_{\text{meta}}} \sum_{k=1}^K \|\mathbf{Z}_{s;k} - \mathbf{V}_{\text{meta}}^\top\|_F^2 \\ &= 2(K\mathbf{V}_{\text{meta}} - \sum_{k=1}^K \mathbf{Z}_{s;k}^\top). \end{aligned} \quad (32)$$

The adapted factor matrix $\mathbf{V}_s$ to each source domain s is expressed as follows:

$$\begin{aligned} \mathbf{V}_s &= \mathbf{V}_{\text{meta}} - \eta \nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}}) \\ &= (1 - 2\eta K)\mathbf{V}_{\text{meta}} + 2\eta \sum_{k=1}^K \mathbf{Z}_{s;k}^\top. \end{aligned} \quad (33)$$

where the first equation uses Equation (13) and the second one is derived from Equation (32). The gradient $\nabla_{\mathbf{V}_s} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$ is expressed in terms of $\mathbf{V}_{\text{meta}}$:

$$\begin{aligned} \nabla_{\mathbf{V}_s} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s) &= 2(K\mathbf{V}_s - \sum_{k=1}^K \mathbf{Z}_{s;k}^\top) \\ &= 2(K(1 - 2\eta K)\mathbf{V}_{\text{meta}} + 2\eta K \sum_{k=1}^K \mathbf{Z}_{s;k}^\top - \sum_{k=1}^K \mathbf{Z}_{s;k}^\top) \\ &= 2(1 - 2\eta K)(K\mathbf{V}_{\text{meta}} - \sum_{k=1}^K \mathbf{Z}_{s;k}^\top). \end{aligned} \quad (34)$$

Thus, the sum of two gradients in Equations (32) and (34) is expressed with $\mathbf{V}_{\text{meta}}$ as the following equation:

$$\begin{aligned} \nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}}) + \nabla_{\mathbf{V}_s} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s) &= \\ 4(1 - \eta K)(K\mathbf{V}_{\text{meta}} - \sum_{k=1}^K \mathbf{Z}_{s;k}^\top). \end{aligned} \quad (35)$$

The gradient $\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$ is computed as follows:

$$\begin{aligned} \nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s) &= \nabla_{\mathbf{V}_{\text{meta}}} \sum_{k=1}^K \|\mathbf{Z}_{s;k} - \mathbf{V}_s^\top\| \\ &= \nabla_{\mathbf{V}_{\text{meta}}} \sum_{k=1}^K \|\mathbf{Z}_{s;k} - (1 - 2\eta K)\mathbf{V}_{\text{meta}}^\top - 2\eta \sum_{l=1}^K \mathbf{Z}_{s;l}^\top\|_F^2 \\ &= 2(1 - 2\eta K) \left((1 - 2\eta K)K\mathbf{V}_{\text{meta}} - \sum_{k=1}^K (\mathbf{Z}_{s;k}^\top - 2\eta \sum_{l=1}^K \mathbf{Z}_{s;l}^\top) \right) \\ &= 2(1 - 2\eta K) \left((1 - 2\eta K)K\mathbf{V}_{\text{meta}} - (1 - 2\eta K) \sum_{k=1}^K \mathbf{Z}_{s;k}^\top \right) \\ &= 2(1 - 2\eta K)^2 (K\mathbf{V}_{\text{meta}} - \sum_{k=1}^K \mathbf{Z}_{s;k}^\top) \end{aligned} \quad (36)$$

where the fourth equality uses $\sum_{k=1}^K \sum_{l=1}^K \mathbf{Z}_{s;l}^\top = K \sum_{k=1}^K \mathbf{Z}_{s;k}^\top$. The proof is finished by comparing Equations (35) and (36). $\square$

A.3 Proof of Theorem 3.2

PROOF. Using Lemma 3.1, the meta-update step of META-P2 in Equation (14) is expressed as follows:

$$\begin{aligned} \mathbf{V}_{\text{meta}} &\leftarrow \mathbf{V}_{\text{meta}} - \eta \nabla_{\mathbf{V}_{\text{meta}}} \sum_{s \in \mathcal{S}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s) \\ &= \mathbf{V}_{\text{meta}} - \eta C \sum_{s \in \mathcal{S}} (\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}}) + \nabla_{\mathbf{V}_s} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)). \end{aligned} \quad (37)$$

Note that $\nabla_{\mathbf{V}_{\text{meta}}} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_{\text{meta}})$ and $\nabla_{\mathbf{V}_s} \mathcal{L}_V(\{\mathbf{X}_s\}; \mathbf{V}_s)$ are expressed by $(\mathbf{V}_{\text{meta}} - \mathbf{V}_s)/\eta$ and $(\mathbf{V}_s - \mathbf{V}_s')/\eta$ using Equations (13) and (17), respectively. Then the meta-update step in Equation (37) is written as follows:

$$\begin{aligned} \mathbf{V}_{\text{meta}} &\leftarrow \mathbf{V}_{\text{meta}} - \eta C \sum_{s \in \mathcal{S}} ((\mathbf{V}_{\text{meta}} - \mathbf{V}_s)/\eta + (\mathbf{V}_s - \mathbf{V}_s')/\eta) \\ &= \mathbf{V}_{\text{meta}} + C \sum_{s \in \mathcal{S}} (\mathbf{V}_s' - \mathbf{V}_{\text{meta}}) \end{aligned} \quad (38)$$

which ends the proof. $\square$

A.4 Proof of Lemma 3.3

PROOF. A naive PARAFAC2 decomposition on source domains takes $O(\sum_{k=1}^K I_k JRS)$ time where S is the number of source domains, and R is the target rank.

The meta-learning-based decomposition of MTD on source domains consists of two steps: 1) performing a typical PARAFAC2 decomposition for each source domain $s \in \mathcal{S}$, and 2) updating the meta factor matrix $\mathbf{V}_{\text{meta}}$. To update the meta factor matrix $\mathbf{V}_{\text{meta}}$, META-P2 first computes $\mathbf{V}_s'$ for each domain $s \in \mathcal{S}$ (see line 13 of Algorithm 1), and merges them (see line 15 of Algorithm 1). Computing $\mathbf{V}_s'$ of domain s takes $O(JKR^2)$ time. Therefore, updating $\mathbf{V}_{\text{meta}}$ by merging $\mathbf{V}_s'$ for $s = 1, \dots, S$ takes $O(JKR^2S)$ time, and the decomposition of META-P2 on source domains takes $O((\sum_{k=1}^K I_k JRS) + JKR^2S)$ time. Since $O(\sum_{k=1}^K I_k) > O(KR)$, the meta-learning-based decomposition of META-P2 on source domains takes $O(\sum_{k=1}^K I_k JRS)$ time, which is the same time complexity as that of a naive PARAFAC2 decomposition on the source domains. $\square$

A.5 Proof of Lemma 3.4

PROOF. Decomposing a new irregular tensor $\mathbf{X}_T$ in a target domain T consists of two steps: 1) initializing $\mathbf{V}_T$ with $\mathbf{V}_{\text{meta}}$, and 2)

performing PARAFAC2-ALS algorithm. Therefore, the time complexity of META-P2 for decomposing a new target tensor is the same as that of PARAFAC2-ALS decomposition on the target tensor. $\square$